

技术交底书

发明创造名称： 基于三层异构架构的张量处理器系统设计		
申请人： 兰州大学 兰州炼芯微架构有限责任公司		
发明人或设计人： 何安平，陈杰，孙一凡，黄渊，陆钊桦，胡蓉青		
申请类别： ☒ 发明 ☐ 实用新型 ☐ 外观设计 （鼠标左键双击相应方框，默认值一栏“选中”）		
联系人：（必填）何安平	电话：（必填）13893178725	E-mail：（必填）heap@lzu.edu.cn
填写注意事项： 1. 申请人可以是个人或/和具有法人资格的企事业单位 2. 发明人或设计人一定是自然人，人数不限。		
备注：		

请认真阅读表格各项的要求，按规定如实、详尽地填写相关内容，可自行增加附页。

1. 发明创造名称（请填写）：基于三层异构架构的张量处理器系统设计

应简单明确地描述发明的实质内容，可按其技术特点、用途或技术和用途双重命名，发明创造名称不得超过 25 个字，避免使用宣传性、广告性词汇。

例如：节水型反渗透净水器

2. 背景技术及现有技术的缺陷和不足：

背景技术：是对最接近的现有技术的说明，可分为大的背景技术和小的背景技术两个方面进行介绍，大的背景技术主要指该技术领域的总体状况，小的背景技术指与本发明改进的具体技术密切相关的技术状况；背景技术的详细程度，以不需要再去查文献即可理解该技术密切相关的技术状况，如果现有技术出自专利文献、期刊、书籍，则提供出处。

张量（Tensor）作为现代科学与人工智能领域的核心数据结构，其定义、应用及运算方案构成了支撑大规模计算的技术基石。在科学计算领域，空气动力学仿真将纳维-斯托克斯方程离散化为高维稀疏张量，需处理海量变量的耦合计算；引力波探测依赖贝叶斯推断，其参数估计本质是高维张量积分问题。人工智能领域中，Transformer 架构的注意力机制依赖三维张量运算，而千亿级参数

的 MoE 模型需动态调度稀疏专家张量。工业仿真场景如气象模型需融合温度/气压等多维气象变量构成的时空张量，3D 物体重建则依赖点云张量的非线性优化。这些应用揭示了张量计算的本质矛盾：物理场仿真中张量维度呈爆炸性增长，而科学计算的超高稀疏性与 AI 训练的稠密-稀疏混合特性，对硬件架构提出异构化需求。张量处理架构的设计目标是通过高效并行化计算和存储优化来提升算力密度与能效比。

在张量计算领域，CPU 凭借冯·诺依曼架构的通用性，成为处理非规则计算的核心载体：其多级缓存（L1/L2/L3）与超标量设计支持细粒度任务调度，避免并行计算中的数据分块空泡；分支预测与乱序执行能力高效处理动态路由（如 MoE 专家选择）和迭代优化算法；大内存容量（128GB+ DDR5）与纳秒级延迟（约 100ns）使其在超大规模稀疏张量（如引力波分析）和实时推理中不可替代。然而，CPU 的并行能力受限于核心数量（≤64 核）与 SIMD 宽度（如 512bit AVX-512），矩阵乘加（GEMM）吞吐量落后 GPU 数十倍（实测 $256 \times 1024 \times 2048$ 矩阵乘法：CPU 耗时 55 秒 vs. GPU 1.3 秒）；内存带宽（DDR5 约 50GB/s）不足 GPU HBM 的 3%，导致卷积等低数据重用操作中带宽瓶颈显著；能效比劣势突出，同等算力下功耗达 200W+，而 GPU 以更低功耗提供更高算力。

GPU 张量核心通过专用硬件设计突破并行计算瓶颈：其 $4 \times 4 \times 4$ 矩阵乘累加（MAC）单元将 16×16 矩阵拆分为 8 线程组并行处理，大幅提升 GEMM 效率；混合精度支持（FP16 乘+FP32 累加）使吞吐量较纯 FP32 提升 8 倍，NVLink 互联（600GB/s）支持多卡集群扩展。统一内存架构（如 Turing 64KB 共享内存）可动态配置为 L1 缓存，减少 DRAM 访问延迟。但高并行计算导致能效比低下（如 V100 达 250W），依赖液冷散热；架构固化使其仅优化固定算子（如 GEMM），对动态规划或非规则稀疏张量需软件预处理；多卡部署需 2 的整数次幂配置，受物理空间与 PCIe 槽位限制。

TPU 以存算一体和极致能效重构张量计算范式：脉动阵列架构预加载参数至计算单元（如 TPUv4 的 32K ALU 直连矩阵），数据移动能耗降至 GPU 的 1/5；专为低精度优化（INT4/INT8），支持稀疏矩阵高效处理，同等任务速度比 GPU 快 2.7 倍（功耗仅 120-150W vs. A100 400W）。但其生态深度绑定 TensorFlow/JAX，对 PyTorch 支持弱；仅限 Google Cloud 部署，缺乏本地化选项；硬件异构性差，难以适配动态分支等非矩阵任务。

算力需求爆发性增长与单板物理极限（空间、散热、通信瓶颈）的尖锐矛盾，暴露单一架构的全面局限性，驱动异构计算架构的发展。传统单一架构（如 CPU、GPU、TPU、FPGA）各有显著短板：CPU 并行不足，GPU/TPU 难适配动态逻辑且高耗，FPGA 计算密度低，无法全面应对从边缘低延迟到千亿级模型训练的全场景需求。单板算力受制于工艺、能效及内存墙，难以支撑日益庞大的张量计算规模。异构架构通过整合各类型计算核心破解困局：其核心是整合异构硬件差异化优势形成系统

级能力——CPU 调度控制流，GPU/TPU 加速稠密运算，FPGA 优化实时任务，辅以资源池化与智能调度。这种“专业硬件精准分工+资源动态协同”的模式，从根本上突破性能瓶颈，在提供高算力密度的同时保障能效与灵活性，成为满足未来智能计算复杂需求的终极路径。

现有技术的缺陷和不足（请填写）：

1. 客观评价现有技术的缺点，会带来哪些问题，这些缺点是通过本技术的发明点来解决的，本技术正是以解决这些问题和缺点为出发点的；本发明无法解决的技术问题不必描述，本发明不能解决的缺点也不必写；

2. 如果找不出对比技术方案及其缺点，可用反推法，根据本发明的优点来找出对应的缺点，还可以从结构角度推导出有先进产品的缺点；

3. 缺点可以是成本高、结构复杂、性能差、工艺繁琐等类似问题。

1. 指令与数据耦合传输的带宽浪费

传统架构（如 GPU 集群）采用指令与数据绑定传输模式。当相同数据需被多个计算单元复用时，必须重复传输完整数据内容，导致传输通道充斥大量冗余信息。这种设计在大规模矩阵运算中尤为低效——例如权重矩阵等高频复用数据反复占用带宽，造成有效通信效率显著降低。

2. 集中式调度的决策延迟

软件中心化调度器（如 Kubernetes）需通过总线多次查询设备状态才能完成资源分配。这种串行化操作产生显著延迟，导致计算单元长期处于空闲等待状态，系统整体吞吐量受限。

3. 数据搬运中断计算

传统数据传输依赖中央处理器调度控制。当数据在计算单元间迁移时，计算操作必须暂停等待搬运完成，造成硬件资源周期性闲置。这种中断显著制约系统效率。

针对前面现有技术的缺点，逐一正面描述本技术所要解决的技术问题（请填写）：

1. 指令/数据分离传输机制

针对指令与数据耦合传输导致的带宽浪费缺陷，本方案建立独立传输通道机制：通过上位机将原始数据预压缩为含唯一 ID 标识的数据包独立下发，同时将计算指令封装为 16 字节标准化格式（操作码+数据 ID+参数）。该设计彻底消除复用数据的重复传输，完全释放通信带宽压力。

2. 元数据驱动硬件调度

针对集中式调度引发的决策延迟问题，本方案构建硬件级实时响应体系：在调度板内置全局元数据表动态追踪三态信息（数据物理位置/子板负载/内存占用），并基于数据本地化原则实施任务分配。该机制使调度延迟降低数量级，资源等待时间显著缩短。

3. 零拷贝双通道流水线

针对数据搬运引发的计算中断缺陷，本方案创建并行传输通道：通过环形缓冲区（FIFO_1-4）建立数据中转站，采用上行数据读取与下行结果写入的双通道流水设计。该技术完全消除数据传输导致的计算停滞，确保计算单元持续运行。

3. 具体的技术方案描述（请填写）：

本部分所提供的内容涉及专利申请文件中最重要的部分，越详细越好，使本专业技术领域中的普通技术人员看到技术描述后能够根据其实施本技术为准。

1. 针对设备/装置/仪器的发明，应当详细、清楚、完整地描述整个装置的结构，具体说明其零部件的形状、结构及其连接关系、空间位置关系、配合关系、工作原理，所起的作用、功能和效果等，尤其是体现本发明构思部分的部件及其连接关系，必要时结合附图加以说明，涉及型号、尺寸、材质等的需说明；

2. 说明装置的使用方法/过程、原理及功能等。

3. 针对方法发明，应当具体说明其详细的工艺方法、工艺流程的步骤和条件（如时间、压力、温度、浓度）、实验效果、数据表格等。

整体架构：

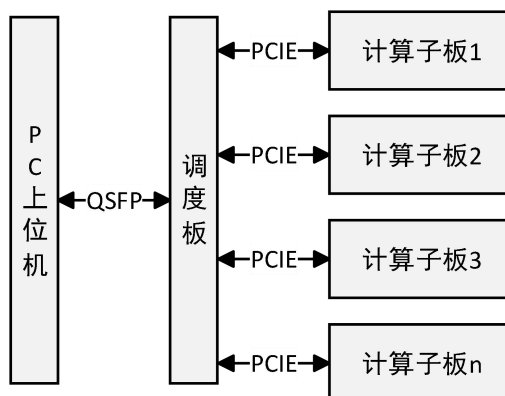


图 1 整体架构图

如图 1 所示，整体采用三层架构设计，由 PC 上位机、调度板以及计算子板阵列共同构成，适用于大语言模型推理和科学计算等高负载场景。其中，PC 上位机部分负责接收计算任务以及数据，并进行初步处理，化为相应的计算任务流以及数据流，下发到调度板，调度板为以 XCVU13P 为核心的 FPGA 板，板载 64GBDDR4、100GbQSFP 和 10GbSFP 光口、多组 PCIE 插槽，提供了强大的数据存储转

发能力，以及丰富的硬件资源，用于实现数据以及计算任务的调度，达到多计算子板负载均衡的效果。计算子板负责处理具体的计算任务，支持多种计算模式，涵盖了向量/矩阵等的多种操作，可以充分满足大语言模型推理以及科学计算等场景下的多种计算需求。

(1) PC 上位机模块

PC 上位机是系统的顶层软件组件，负责接收原始计算任务、拆解任务、生成指令、处理数据传输，并整合最终结果。它通过 QSFP 接口与底层调度板通信。

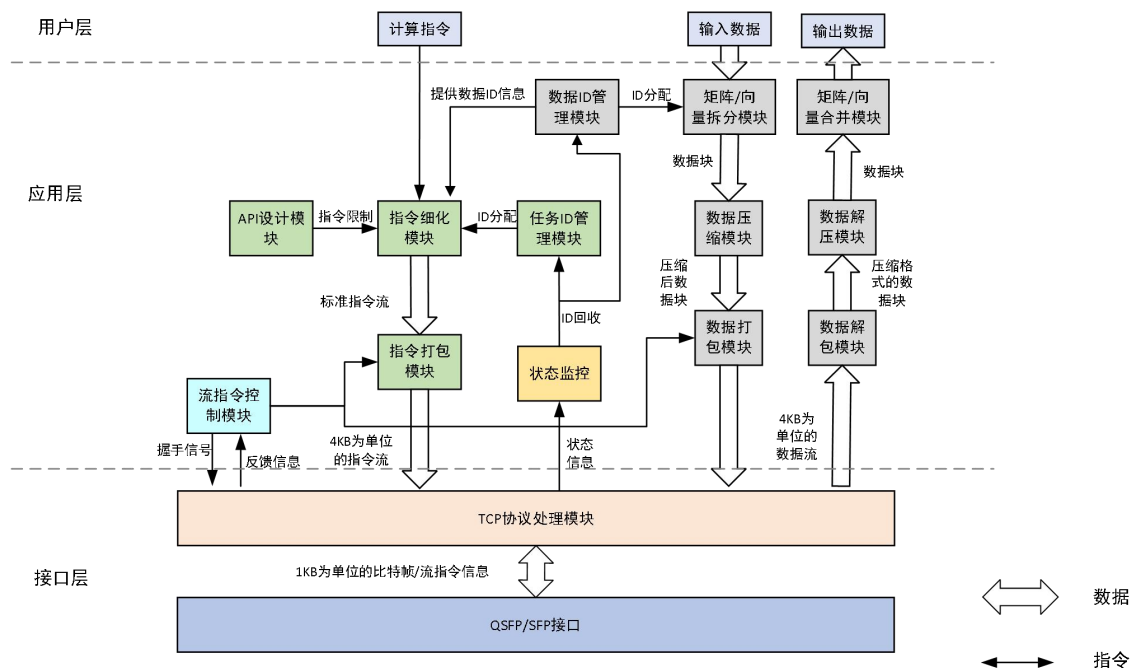


图 2 PC 上位机架构图

① 输入输出：

上位机部分的输入输出分为两个部分，一个是针对用户接口的输入输出，主要集中于用户层部分，另一个是针对调度板的输入输出，主要集中于接口层部分。其中，用户层部分的输入为原始计算任务及其数据，如大规模矩阵运算（例如 2048x2048 矩阵乘法）、科学计算任务或大语言模型推理任务，输入数据包括矩阵数据、任务参数（如维度、位宽）等。输出为完整计算结果，如合并后的稠密矩阵、向量或模型输出。输出通过结果合并模块生成，并可用于上层应用。接口层的输入为调度板返回的结果数据包以及状态监控信息、握手信号反馈等。输出为握手信号、输入数据包、计算任务包等。

② 模块设计：

i. ID 管理模块：

负责任务/数据的 ID 分配与回收，实现任务/数据的唯一性管理，避免数据/任务混乱。任务和

数据 ID 管理方案一致，具体为：在 PC 上位机部分维护两张表，表以栈的形式存在，一张是可用 ID 表（未分配 ID），一张是已释放 ID 表，ID3 字节，初始化时，可用 ID 表满，已释放 ID 表空，然后每生成一个分块/任务，就递增分配一个可用 ID 给块，一直到完全分配完 ID，可用表栈堆为空。与此同时，被释放的数据块/已完成任务的 ID 被装入已释放 ID 表，插入该表。当可用 ID 表空时，将已释放 ID 表作为新的可用 ID 表，旧的可用 ID 表置空，作为新的已释放 ID 表。重复分配过程和释放过程，实现最多 16M 个唯一 ID 的维护，表的大小为 3 字节*16M。

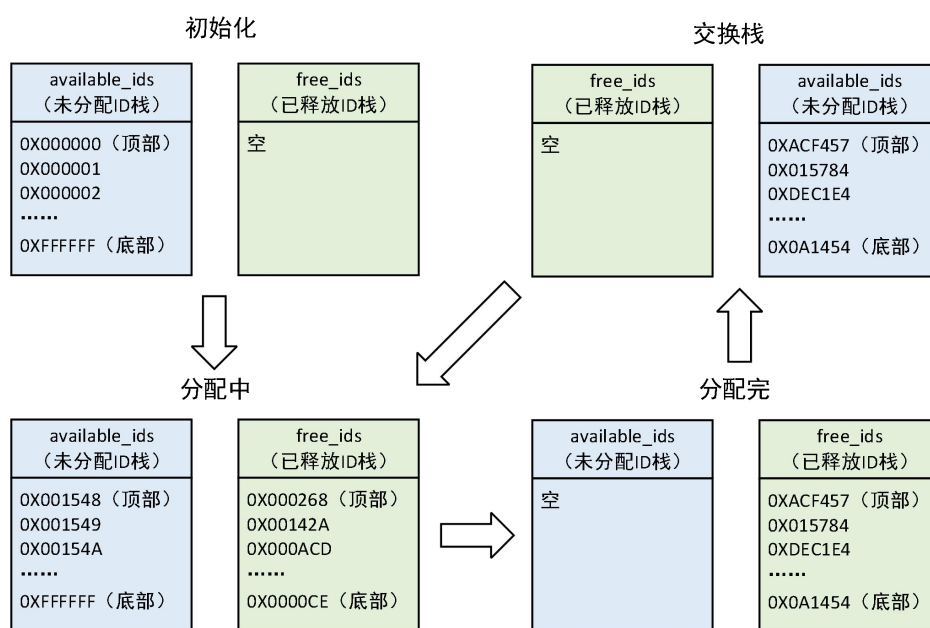


图 3 ID 管理流程图

ii. API 设计模块：

该模块通过统一计算指令格式，实现了跨硬件层的高效协同，既屏蔽了调度板与计算子板的结构性差异，同时使得开发者专注于指令的使用，无需关注 QSFP/PCIE 等接口传输协议的细节。这种设计不仅通过任务 ID 实现全链路状态追踪和错误控制，还预留了操作码扩展空间以适配未来需求，成为支撑三层架构下任务拆解、调度与合并的核心枢纽，确保了大语言模型推理等高负载场景的系统性能。

指令码格式如下：

[任务 ID:3B | OP 码:1B | 输入 A:4B | 参数:4B | 输出 C:4B]

指令统一为 16B 长，其中前 3B 为任务唯一 ID，在生成计算任务时由任务 ID 管理模块进行分配，在任务完成后释放该 ID，下一 B 为操作的 OP 码，目前定义了 21 种操作 OP，1B 的宽度使得日后还拥有足够的宽度用于扩展 OP 数目。剩下 12B 分别为输入 A 的 ID、输入 B 的 ID 和输出 C 的 ID，在仅

有单输入的情况下，B 还可作为常量参数或空置。如在激活函数运算中，B 标志着激活函数类型，在标量相关运算中，B 用于存储标量而不是 ID。详细 OP 码如下表所示：

表 1 OP 码

矩阵计算模式	OP 码	计算类型说明
矩阵×矩阵=矩阵	0x01	标准矩阵乘法
矩阵+矩阵=矩阵	0x02	矩阵逐元素加法
列向量×行向量=矩阵	0x03	外积运算
矩阵×列向量=列向量	0x04	矩阵右乘列向量
行向量×矩阵=行向量	0x05	行向量左乘矩阵
行向量×列向量=数	0x06	内积运算
数×向量=向量	0x07	标量向量乘法
数×矩阵=矩阵	0x08	标量矩阵乘法
向量按位乘法	0x09	Hadamard 乘积
向量内部求均值	0x0A	向量元素均值计算
向量加减法	0x0B	向量逐元素加减
向量逐元素除法	0x0C	向量元素级除法
向量逐元素乘法	0x0D	向量元素级乘法
激活函数	0x0E	非线性函数映射
矩阵转置	0x0F	矩阵行列转置
矩阵对角化	0x10	提取/生成对角矩阵
向量对角化	0x11	向量转对角矩阵
向量合并	0x12	向量拼接操作

矩阵合并	0x13	矩阵块拼接
矩阵转向量	0x14	矩阵展开为向量
矩阵行置换	0x15	矩阵行交换操作

iii. 流指令控制模块：

流指令控制模块负责与调度节点进行通信，通过握手协议实现数据包传输、计算任务调度及状态反馈。流指令分为五类，每类指令均包含操作码（OP）和参数，并返回执行状态。

表 2 指令 OP 码汇总表

操作码	指令名称	功能描述	负载格式	响应码	备注
0xA0	数据发送指令	协商数据传输	[0xA0][ID1(4B)]. ..[IDn(4B)]	0x01: 接收准备就绪 0x02: 内存不足需释放	需主动释放存储资源
0xA1	数据请求指令	主动获取数据	[0xA1][ID1(4B)]. ..[IDn(4B)]	0x03: 全部数据可用 0x04: 部分数据缺失	异常处理：需查询缺失 ID 列表
0xA2	数据释放指令	回收存储资源	[0xA2][ID1(4B)]. ..[IDn(4B)]	0x05: 全部释放成功 0x06: 部分释放失败	异常处理：返回失败 ID 的锁定状态
0xA3	计算指令发送	提交计算任务	[0xA3]包数(2B)]	0x07: 计算单元就绪 0x08: 资源过载	流控机制：启动指数退避重试策略
0xA4	状态反馈指令	获取系统健康	[0xA4]	0x09: 返回子系统状态 (含状态编码)	状态编码： 0x00: 正常 0x01: 警告 0x02: 故障

该设计确保流指令控制模块高效、可靠地管理数据传输与计算任务调度。

iv. 指令细化模块：

该模块主要负责将用户传入的计算指令细化为硬件可执行指令，如：用户发来一个全连接层计算，指令细化模块会获取注入的权重矩阵和待计算向量的数据块 id 信息，然后结合数据块 id，将一次全连接层计算化为详细的以数据块为单位的矩阵向量乘和向量加操作等。然后将对应操作转化为 API 设计模块中设计的标准指令模式，同时为这些指令附上唯一任务 id，以便追踪其任务状态。

v. 数据拆分/合并模块：

该模块负责将大规模的矩阵/向量数据拆分为固定的 256 大小的数据块并为每一个数据块赋予唯一数据 id。便于指令细化模块根据数据 id 定位到对应的数据块，完成计算指令的生成。在回收结果时根据原始数据中记录的数据块 id 信息，将结果数据填入对应的区域，完成相应的数据块的合

并过程，得到完整的结果块发送给用户。

vi. 数据压缩/解压模块：

数据在分块后会进入数据压缩模块，采用 COO 格式对数据块 进行压缩，剔除其中的 0 元素，只保留非 0 元素，以降低数据传输的通信开销。在收回结果数据时，结果数据仍然是 COO 格式的压缩数据，此时就需要对该数据进行 COO 解压缩，还原出最初的数据块形状和格式，以便于数据合并模块能够得到完整的数据块进行合并。

vii. 打包/解包模块：

单个数据块大小过大且压缩后大小不规则，导致使用 PCIE 进行数据传输时，无法以满速率进行数据传输，因此将数据块进一步拆分为 4KB 大小的数据包的集合，传输数据时以 4KB 为单位进行传输。但是上位机和调度板之间以 TCP 协议进行传输，TCP 协议单次最大传输字节为 1448 字节，为便于数据传输，会在 TCP 协议处理模块处将 4KB 包进一步拆分为 1KB 的数据帧，上位机和调度板之间的数据流传输以 1KB 大小的数据帧为单位进行。

viii. 状态监控模块：

状态监控模块有两个主要功能，一方面，状态监控模块要处理调度板传回的状态信息，包括各个计算子板和调度板的资源使用情况以及运行状况，流指令控制模块会根据该状态反馈信息决定后续的数据传输。另一方面，状态监控模块会将调度板传回来的任务信息以及数据块信息进行解析，若任务完成或数据块释放成功，则会将完成的任务 id 和数据块 id 分别传回到两个 id 管理模块，进行 id 的释放。

③ 工作流程：

上位机部分的工作流程主要分为 4 个部分：数据预处理阶段、数据传输阶段、计算执行阶段、结果回收阶段。下面将详细描述 4 个工作阶段。

i. 数据预处理阶段：

在此阶段中，上位机接收用户传来的输入数据信息，然后对输入数据进行拆分、编码、压缩、分包等一系列流程，将输入的纯数据形式的大规模数据拆分为一个个以 1KB 为基础单位的数据帧，以待后续传入调度板进行处理。处理流程如图 4 所示：

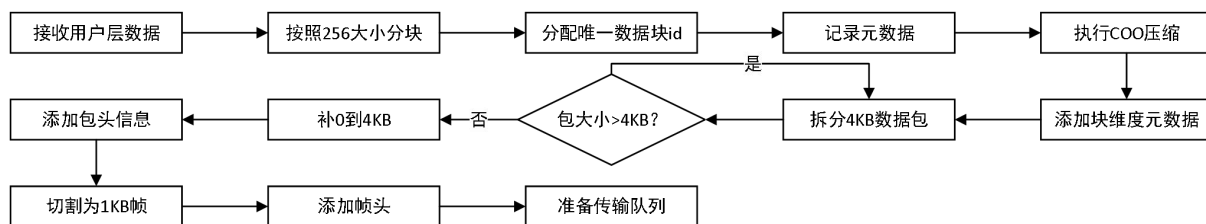


图 4 数据预处理流程

数据预处理流程开始于接收来自用户层的原始输入数据。系统立即执行分块操作，将数据按照固定尺寸 256 进行分割，对于矩阵采用 256×256 规格，对于向量则采用 256×1 规格划分。每个新生成的数据块会被分配唯一的标识 ID，这个全局唯一的块 ID 将与数据块的原始位置信息建立映射关系，完整记录在专属的元数据记录系统中。

接下来流程进入压缩阶段，对每个数据块实施 COO 格式压缩处理。在压缩工作完成后，系统会为压缩块添加关键维度元数据，详细描述数据类型和非零元素数量等特征信息。随后流程进入判断环节，系统会自动检测当前压缩块的大小，当数据量超过 4KB 阈值时，会执行拆分操作，将数据切割为标准的 4KB 数据包单元。对于长度不足 4KB 的数据包，系统自动补零直至达到标准尺寸。

随后所有数据包单元都会进行统一规范处理。为每个标准包都添加包含关键信息的包头，标注所属数据块 ID、数据包总数、当前包序号以及包状态等核心参数。完成包封装的数据被进一步切割为更细粒度的 1KB 数据帧，以适配 TCP 传输协议的要求。

最后阶段为每个数据帧添加精炼的帧头信息，明确标识所属数据块 ID、包序号和帧序列号等关键索引。所有帧按照严格的顺序规则进行有序排列，形成结构完整的传输队列，确保数据传输的稳定性和完整性。整个预处理体系逐级递进，形成标准化的数据传输准备流程。

ii. 数据传输阶段：

在完成数据的预处理之后，进入数据的传输阶段，在该阶段中，上位机和调度板通过握手协议的方式，确保数据能够完整成功发送到调度板中。具体工作流程如图 5 所示：

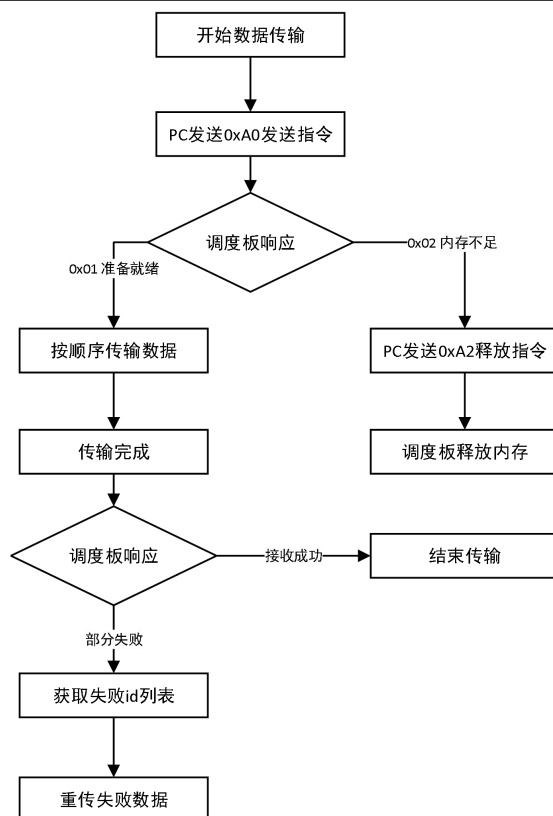


图 5 数据传输流程

系统数据传输流程从 PC 向调度板发送 0xA0 数据发送指令开始。此指令附带数据块 ID 列表，表示准备传输的数据内容。调度板接收指令后返回响应码：当响应为 0x01（准备就绪）时，PC 立即启动数据帧传输过程。来自预处理阶段的有序传输队列按照严格顺序发送至调度板。当响应为 0x02（内存不足）时，PC 则发送 0xA2 数据释放指令，要求调度板释放特定数据块空间。完成内存释放后，PC 重新发送 0xA0 指令，循环此过程直至获得准备就绪响应。

成功传输所有数据帧后，PC 等待调度板最终状态反馈：如收到完整接收成功的响应，则结束传输流程。如收到部分数据失败的响应，PC 获取具体失败数据 ID 列表，仅针对失败数据重新发起传输，直至调度板确认全部接收成功。

整个流程建立了闭环重传机制，确保数据零丢失传输。传输顺序严格遵循预处理阶段形成的队列结构，保障时序一致性。重传过程仅针对失败数据块实施，最大化传输效率。

iii. 计算执行阶段：

在传输完数据后，会进入计算指令执行阶段，在该阶段中，会接收来自用户层的计算指令，然后对其进行基于数据块的计算指令细化操作，在完成细化后将计算指令发送到调度板，以完成计算任务。具体处理流程如图 6 所示：

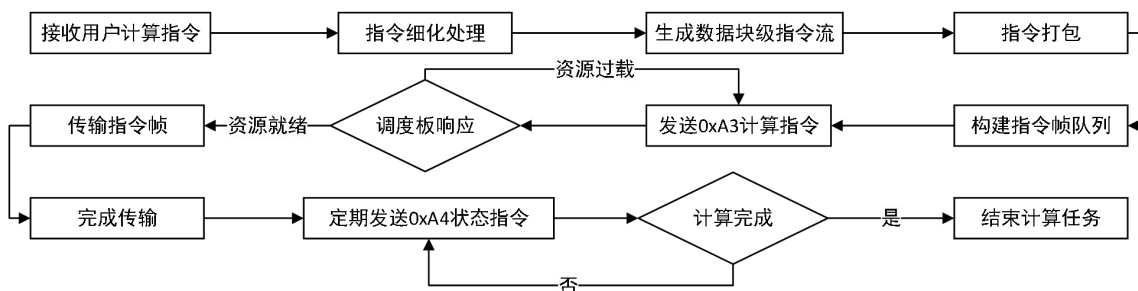


图6 计算执行流程

计算执行流程始于系统接收用户层的计算指令，随即进入指令细化模块进行分解处理。该模块基于数据块的固有划分结构，将用户指令拆解为多个原子级的操作单元，每个单元严格遵循 16 字节的标准指令格式以适应硬件执行要求。典型的细化操作包括将复杂矩阵乘法转化为向量乘加序列，或将卷积运算分解为滑动窗口计算等核心处理步骤。

细化后的指令流进入封装与分包阶段，系统将连续指令序列按 4KB 为单位切割成标准指令包，每个包正好容纳 256 条 16 字节指令。完成分包后，每个 4KB 指令包被进一步切割为 4 个 1KB 传输帧，帧头携带包 ID 和帧序号等关键元数据，由此形成的传输队列完全匹配数据传输阶段的封装规范。

准备就绪后系统启动指令传输握手流程，PC 向调度板发送 0xA3 指令并携带包数参数。调度板返回的响应将决定后续操作：0x07 资源就绪状态将触发指令帧传输流程，而 0x08 资源过载状态则启动指数退避策略，初始等待 100ms 后逐次倍增延迟时间。该握手过程设置 5 次重试上限，超过后即上报计算拒绝错误。

获得资源就绪响应后 PC 即按顺序发送指令帧队列，传输期间实时监测流控状态。当检测到 TCP 层异常丢包时，系统立即触发当前包重传机制确保完整性。所有指令包传输完成后系统等待调度板发回最终接收确认，为后续计算提供保障。

指令传输完毕即进入状态监控阶段，系统以 500ms 为间隔周期性地发送 0xA4 状态查询指令。每次接收响应后解析关键信息：任务状态标记区分进行中（0x00）与完成（0x01）状态；子板负载指数量化 0-100% 的资源压力；错误代码则标志特定异常如计算超时（0x02）。当系统检测到所有任务状态标记转为完成时，正式结束计算执行流程。若超过预期时间 300% 仍未完成，则启动异常中断协议处理机制，通过闭环监控保障任务可靠执行。

iv. 结果回收阶段：

在完成计算指令后进入结果回收阶段，该阶段中，PC 会向调度板发送数据请求指令，从调度板处传回完整的结果数据，在确认结果无误后，进行所有数据的释放操作，包括输入数据、临时数据、结果数据等，完成整个计算流程。结果回收阶段的工作流程如图 7 所示：

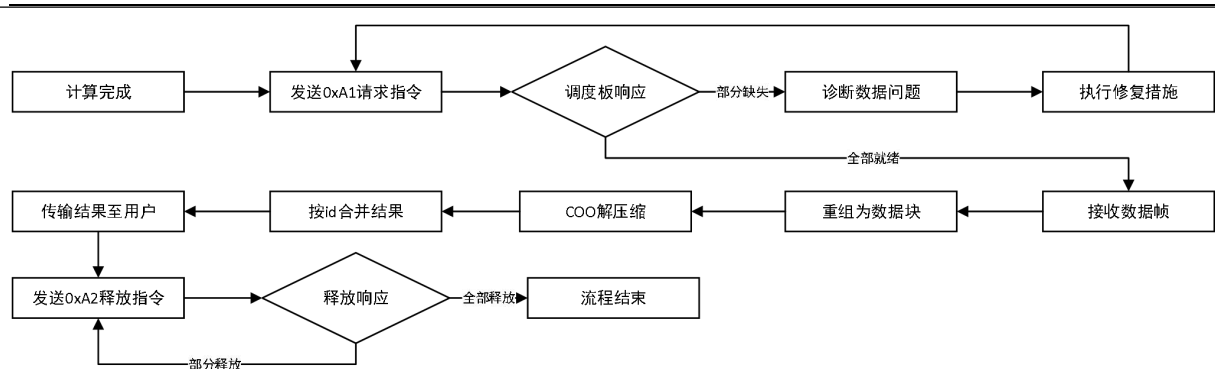


图7 结果回收流程

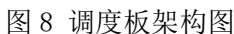
计算任务完成后系统立即启动结果回收流程，PC 首先向调度板发送 0xA1 数据请求指令，明确请求结果数据块的 ID 列表。调度板返回响应状态是关键决策点：当响应为 0x04 表示部分数据缺失，系统进入诊断程序检查具体问题根源（计算错误或存储异常），针对不同情况执行数据修复或重新计算措施，然后重新发送数据请求指令；当响应为 0x03 表示所有数据准备就绪，系统随即开始接收调度板传输的数据帧序列。

接收到数据帧后系统执行多级重组操作：按帧头包含的包 ID 和帧序号信息将 1KB 帧组合为 4KB 标准数据包，再依据包头中记录的块 ID 信息将数据包重组为完整数据块。重组完成的数据块进入解压缩流程，通过 COO 逆向转换恢复原始数据结构，包括复原矩阵/向量的维度和填充零元素，最终按照数据预处理阶段建立的 ID-位置映射表将各数据块合并为完整结果，并返回至用户端。

结果交付后系统立即发起资源回收指令，向调度板发送 0xA2 释放指令要求清除所有计算数据。调度板返回的释放响应状态决定后续操作：0x05 成功状态直接结束全流程；0x06 部分失败状态则需处理锁定数据问题（如因系统保护机制无法释放的内存），解决锁定问题后重新发送释放指令。该回收阶段形成资源管理闭环，确保计算资源高效回收利用。

(2) 调度板模块

调度板是系统的中枢硬件组件，负责接收 PC 上位机的指令、解析任务、管理资源、调度子板任务，并返回结果。它通过 QSFP 接口与 PC 上位机连接，并通过 PCIE 接口与 4 个计算子板交互。



① 输入输出:

调度板作为系统的核心协调中枢，其接口设计围绕流控制指令和数据包两类信息流展开。在输入侧，来自上位机（PC）的 QSFP 接口主要承载两种数据：一类是核心流控制指令，具体包括数据发送（0xA0）、数据请求（0xA1）、数据释放（0xA2）、计算指令发送（0xA3）及状态反馈（0xA4）五种标准指令，用于协调资源管理与计算调度；另一类是纯粹的封装数据包，其中包含预处理后的数据包以及细化的计算指令包，统一以 1KB 帧结构传输。同时来自计算子板的 PCIe 输入也包含两类关键内容：流控制指令的响应信息，例如数据就绪状态反馈或内存操作结果；以及需回传的压缩格式结果数据包，以标准包结构组织，统一以 4KB 包结构传输。

调度板的输出接口延续对称设计逻辑。向上位机方向通过 QSFP 发送的包含流控制指令的响应结果，如数据接收确认、资源释放报告或系统健康度聚合状态，以及需要回传至用户的结果数据包。向计算子板方向通过 PCIe 传输的则包括下发的流控制指令，如数据缓存通知、任务执行指令或状态查询命令，并转发需子板处理的纯粹数据包，这些数据包内封装细分的计算任务指令或待处理的原始数据块。整个系统以 4KB 数据包作为统一载体，无论指令响应还是数据载荷均遵循相同的封装协

议，实现指令流与数据流的高效整合与无差异传输。

② 模块设计：

i. 流指令控制模块：

该模块通过 QSFP 接口接收上位机传输的 1KB 帧格式指令流。协议处理层完成 TCP 解包后，指令数据直接进入控制核心进行解析：针对数据操作类指令（0xA0/0xA1/0xA2），模块提取数据块 ID 并查询 DDR 内存中的 ID 映射表，确定目标子板编号后将指令转发至对应 PCIE 通道；针对状态指令（0xA4）则向各子板广播状态请求，接收子板状态数据流后与调度板自身状态拼接成完整响应包；计算任务下发指令（0xA3）触发调度器模块检查各子板状态，同时向上位机发送就绪响应，启动 4KB 计算指令包接收序列。

ii. 调度器模块：

承担双重调度职能：在计算任务处理维度，接收来自任务队列 FIFO 的指令包后实施双层优先级调度。首先基于任务属性（紧急度/依赖关系）排序任务优先级，再根据子板实时负载数据（通过状态监控模块获取）与数据局部性特征，动态计算各子板的任务适配度，最终将任务分派至最优子板。在数据迁移维度，当检测到跨子板数据依赖时，自动生成 DMA 迁移指令，经由流指令控制模块启动 PCIE 通道间的数据搬移流程，迁移数据暂存于环形缓冲区并更新 ID 映射表状态标志。

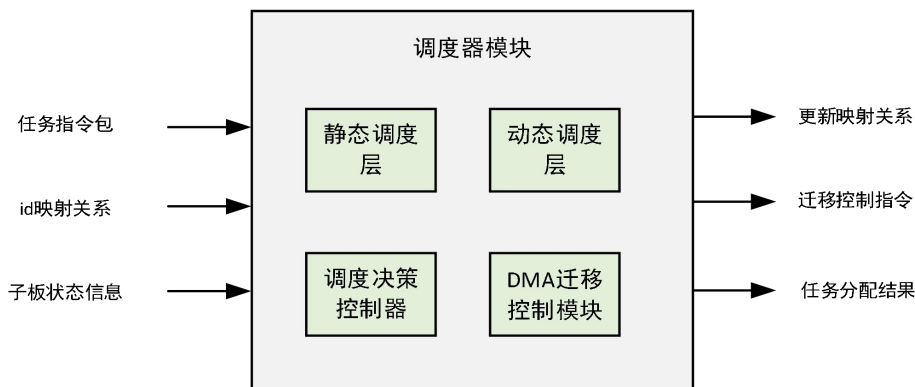


图 9 调度模块示意图

静态调度层核心机制基于任务依赖图解析与紧急响应需求。系统自动解析张量计算任务的依赖拓扑，按数据依赖关系构建执行序列——无依赖的独立任务优先进入就绪队列，多级依赖任务则等待前序节点完成。允许高优先级任务（如实时推理请求）直接抢占当前队列，通过专用通道实现优先分配，确保关键计算任务立即获得调度窗口。该层维持轻量级优先级标记机制，仅区分常规任务与可抢占任务两级，避免复杂分级带来的管理开销。

动态调度层依据硬件实时状态分配计算资源，核心优化目标为张量计算效率最大化。系统持续

监测各计算子板的运行负载（计算核心利用率/内存占用量），将新任务动态分配至负载最低的目标子板实现负载均衡。同时分析张量数据分布特征：若任务所需张量已缓存在某子板显存中，则直接锁定该子板执行以消除数据搬迁成本；对于需跨子板迁移的大型张量，自动评估 PCIE 通道带宽利用率并选择传输代价最小的路径。

两级调度形成高效的协同机制：静态层通过依赖解析保障计算正确性，并为紧急计算任务开辟快速通道；动态层依托实时负载与数据位置感知，使张量计算任务始终贴合最优硬件资源。当动态调度层检测到子板内存不足或算力过载时，立即反馈至静态层重新编排任务序列，防止计算流水线阻塞。

iii. 状态监控模块

该模块持续处理来自子板的 PCIE 状态数据流。对于周期性的 0xA4 指令响应，实时解析子板传来的状态信息：子板计算负载率（0-100%量化值）、内存占用指标、链路误码率，并加入调度板自身状态信息。生成的状态报告同时分发给两个核心模块：向流指令控制模块提供用于上位机响应报文；向调度器模块传递子板状态信息辅助调度决策。当检测到子板故障码（如 0x02 超时）时，立即触发调度器的任务重分配协议。

iv. 协议处理模块：

双通道协议引擎采用差异化处理策略：上位机侧 QSFP 通道以 1KB 帧为基本传输单元，接收数据先存入 16KB TCP 缓冲池，流控制指令则直接转发至流指令控制模块；子板侧 PCIE 通道以 4KB 包为传输单元，经 16KB 缓冲池后按 ID 映射表路由。核心在于流控制指令的快速通道机制——所有指令帧绕过缓冲区直接送入流指令控制模块，进行实时处理。物理层通过 XDMA ip 核实现 PCIE 数据包与 AXI 总线的双向转换，保证带宽利用率。

v. DDR 内存模块

板载 64GB DDR4 内存资源（32bank×2GB），划分为四个功能分区：

数据 ID 映射表：占用 64MB 连续空间，采用直接寻址模式（3 字节数据 ID 作为索引偏移）。每 4 字节表项包含：31 位有效标志、30-27 位目标子板 ID、26 位缓冲区状态位、23-0 位环形缓冲区起始地址。支持唯一写操作者调度器模块确保映射表不会因写冲突导致混乱。

表 3 id 映射表字段解析

比特位	字段	说明
31	Valid	启用标志(1:有效)
30-27	Subboard	目标子板 ID(0-3)，可以存两个子板 ID

26	Buffer_Flag	缓冲区状态(1:使用中)
25-24	Reserved	保留位(置 0)
23-0	Buffer_Addr	缓冲区起始地址 (Buffer_Flag=1 时有效)

ID 映射表作为调度板的核心元数据枢纽，初始化为空状态，当上位机通过 QSFP 下发数据包时，对首次出现的 ID 基于负载均衡选择目标子板并建立永久映射，对已注册 ID 则直接路由至对应子板；该表集成双重定位能力，除记录 ID 与子板映射外，还通过专用状态位标识缓冲区状态——子板间迁移时数据暂存于环形缓冲区，映射表同步更新缓冲区地址及启用标志；资源回收阶段响应 0xA2 释放指令，依据映射定位数据物理位置（子板/缓冲区）执行定向清除并移除条目，实现低延迟跨子板数据迁移与存储资源闭环管理。

数据缓冲区：上行/下行方向分别部署 4 组深度 FIFO（对应子板编号），缓冲区 FIFO 累计满 4 KB 数据包即触发 DMA 搬运。上行 FIFO 缓存子板结果数据包，下行 FIFO 暂存待下发数据块。

任务队列 FIFO：缓存上位机下发的计算指令包，累计达 4KB 或超时唤醒调度器取指流程，确保指令流持续供给。

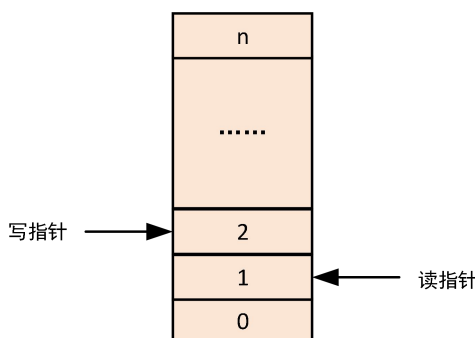


图 10 FIFO 示意图

4×环形缓冲区：每个环形缓冲区以 4KB 为基本单元支持子板间直接数据迁移，源子板写入后置位 Buffer_Flag 并记录地址，目标子板通过 ID 映射表定位读取。写入数据时采用顺序写入，写入满后自动从头开始覆盖，确保数据不会长时间固定在调度板上。在环形缓冲区内，写指针顺序写入数据，读指针则根据 id 映射表中的地址信息，根据地址信息乱序读取数据而不是顺序读取。

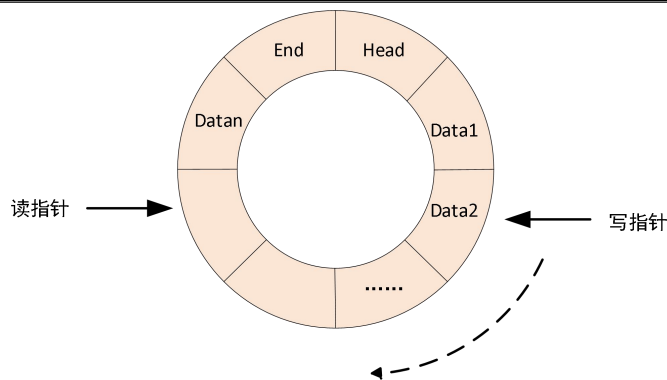


图 11 环形缓冲区示意图

③ 工作流程：

i. 数据接收与预处理：

这一阶段主要处理接收上位机传来的输入数据的接收、分类与初步分发。通过流指令控制模块解析到的不同指令，对不同类型的数据进行针对性处理，转移到不同的存储空间内。具体工作流程如图 12 所示：

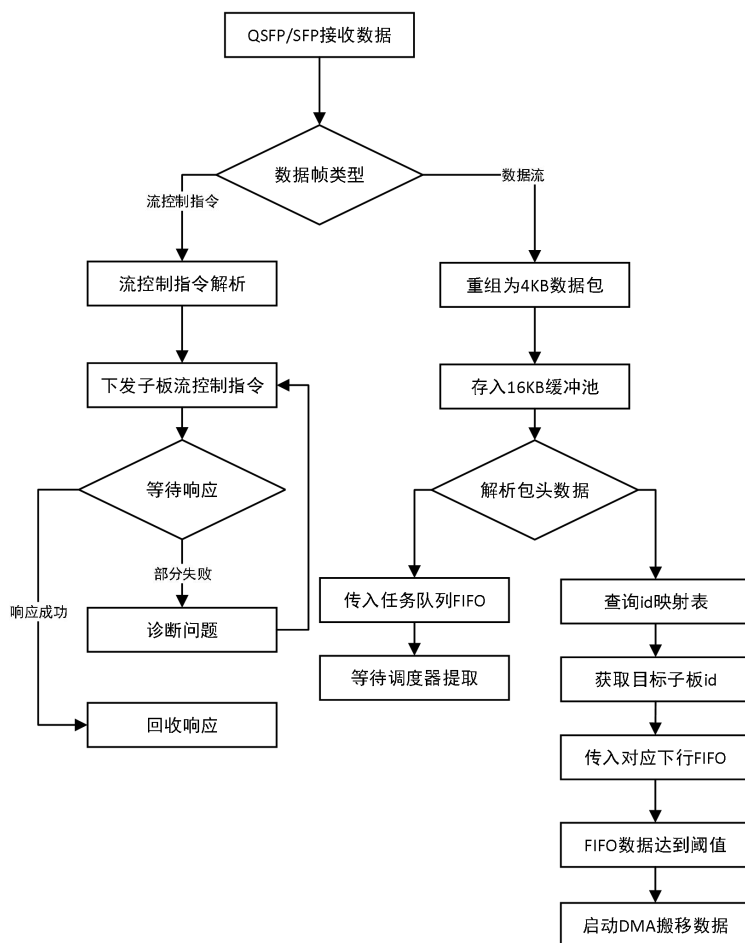


图 12 数据接收与预处理流程图

当调度板通过 QSFP/SFP 接口接收上位机传来的数据后，首先进行数据类型判断流程。若识别为流控制指令（包括 0xA0 数据发送、0xA1 数据请求、0xA2 数据释放等指令类型），则直接进入流指令控制模块进行实时解析。该模块立即处理指令内容，将需要下发至子板的控制信号转发至目标子板，同时启动响应等待机制。当接收到子板返回的成功响应（如 0x01 接收就绪、0x03 数据就绪等）时，调度板将对应响应码回传上位机，结束该指令处理周期；若收到部分失败（0x04/0x06）或失败响应（0x02/0x08），则触发诊断协议分析失败问题，并将诊断报告整合至响应报文返回上位机，等待上位机决策介入。

对于识别为数据流的传输内容（包括压缩数据包和计算指令包），则启动数据处理流水线。协议处理层首先执行数据帧重组：基于序列号校验和 CRC 校验，将离散的 1KB 数据帧重构为 4KB 完整逻辑包，存入 16KB 输入缓冲池暂存。随即 AXI_DMA 控制器解析包头元数据：若为计算指令包（OPCODE=0xA3），整包推送至任务队列 FIFO，等待调度器提取任务；若为普通数据包，则查询全局 ID 映射表——首次出现的数据 ID 自动注册并基于负载均衡算法分配目标子板（0~3 编号），已注册 ID 则直接获取目标位置。两种情况下数据均路由至对应子板的下行数据 FIFO 通道（FIFO_1~FIFO_4）。

每个下行 FIFO 通道独立监控数据累计量，当达到预设 128KB 阈值时，激活 PCIE 传输。此阶段采用 XDMA（Express DMA）硬件加速技术，通过地址转换表（ATS）将数据包物理映射至目标子板内存空间，执行零拷贝直写操作，实现高效数据传输。整个处理流程形成从接收判断→指令直通/数据重组→智能路由→阈值触发的完整闭环，在保障吞吐率的同时实现指令与数据的隔离处理优化。

ii. 计算调度与执行

在完成数据的传输后，调度器在接收到任务队列 FIFO 有值时会提取计算任务开始调度，对于部分需要在不同子板间搬移数据的计算指令，会统计需要搬移的数据，然后在计算指令调度完成后开始进行数据的搬移，在搬移结束之后下发计算指令到相应子板，由子板开始执行计算任务。具体工作流程如图 13 所示：

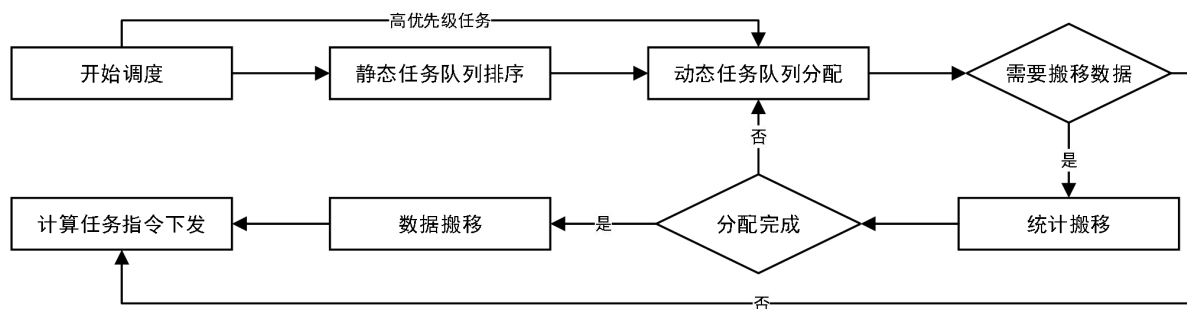


图 13 计算调度与执行流程图

计算任务调度流程启动后。系统首先执行关键任务分流操作：所有进入调度器的高优先级任务

均直接输送至动态调度层进行即时分配。普通任务则按照预设规则进入静态调度层，该层通过分析任务之间的相互依赖关系，构建严谨的执行序列拓扑结构。静态调度依据数据依赖链与计算依赖链双重维度，对所有任务进行排序重组，确保前序计算结果可被后续任务直接复用，形成高效的就绪任务队列后将其移交至动态调度层。

动态调度层作为核心决策中枢，实时监控各计算子板的多维状态指标：包括核心利用率、内存带宽占用率以及 PCIE 链路质量等关键参数。同时综合评估任务预期执行时间和数据局部性特征，为每个任务分配合适的计算子板。任务分配决策实施后，系统启动迁移判定机制：对于数据已驻留在目标子板的任务，直接归类至无需迁移任务组；对于存在跨子板数据依赖的任务，则记录源子板与目标子板的对应关系。

流程进入循环判定阶段：系统持续检查任务分配完成状态。若检测到尚未分配完毕的任务，则返回动态调度层重新执行分配逻辑；当所有任务均完成子板分配时，系统首先批量下发无需数据迁移的任务组至对应计算子板，立即启动计算执行流程。

与此同时，对于存在数据迁移需求的任务组，调度器触发数据搬移引擎：根据先前统计的迁移路径信息，调度数据从源子板经 PCIE 通道精准传输至目标子板。在确认所有数据迁移工作完成后，最终将关联的计算任务正式下发至目标子板。整个流程通过“分配-迁移-下发”的三阶闭环设计，既保障高优先级任务的即时响应能力，又确保复杂依赖任务的严谨执行序列，最终实现计算资源的最大化利用。

iii. 状态监控与资源回收：

计算任务启动后，上位机周期性发送状态请求至调度板。调度板广播查询各子板运行状态，聚合解析负载、进度等关键指标后，封装为状态帧回传。此监控循环持续至任务完成。任务结束后，上位机取回结果数据并发送释放指令。调度板联动子板清除内存数据，同步回收映射表条目及缓冲区资源，为后续计算初始化存储环境，形成资源闭环管理。具体流程如图 14 所示：

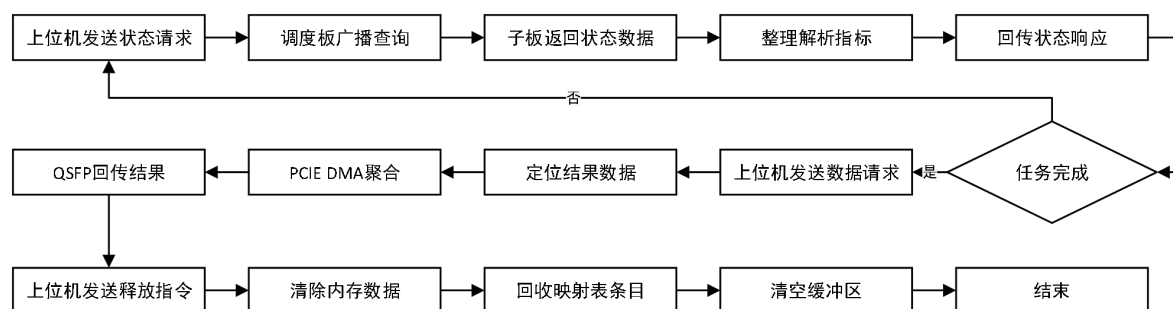


图 14 状态监控与资源回收流程图

在计算任务启动后，上位机系统将自动激活周期性状态轮询机制。系统以预设时间间隔（默认

500ms) 向调度板发送状态请求指令, 调度板接收指令后立即向所有计算子板广播状态查询信号。各子板响应包含核心运行指标: 计算核心实时负载率、内存占用峰值、任务执行完成情况及潜在错误代码。调度板聚合四路子板响应数据, 通过状态解析提取关键参数, 最终封装为统一格式的状态报告帧, 经 QSFP 接口回传至上位机。循环持续运行, 直至捕获到所有子板返回的任务完成标志位。

任务完成信号触发后, 系统进入结果回收阶段。上位机首先发送数据请求指令, 调度板依据全局 ID 映射表定位结果数据物理位置, 通过 PCIE DMA 通道将分布式存储的计算结果聚合重组, 经 QSFP 链路完整回传。数据交付完成后立即启动资源回收协议: 上位机下发数据释放指令, 调度板解析目标 ID 列表, 联动各子板执行内存释放操作, 同步清除环形缓冲区暂存数据及映射表相关条目。回收过程实施严格验证机制, 通过 DDR 控制器校验地址空间归零状态, 确保 100%资源释放成功率, 为后续计算任务初始化纯净的存储环境, 构建起“监控-回收-复用”的资源管理闭环。

(3) 计算子板模块

子板是系统核心执行单元, 负责接收调度板的指令和数据, 执行数据接收、资源管理、调度计算及状态返回等任务。它通过 PICE 接口与调度板进行数据交换。

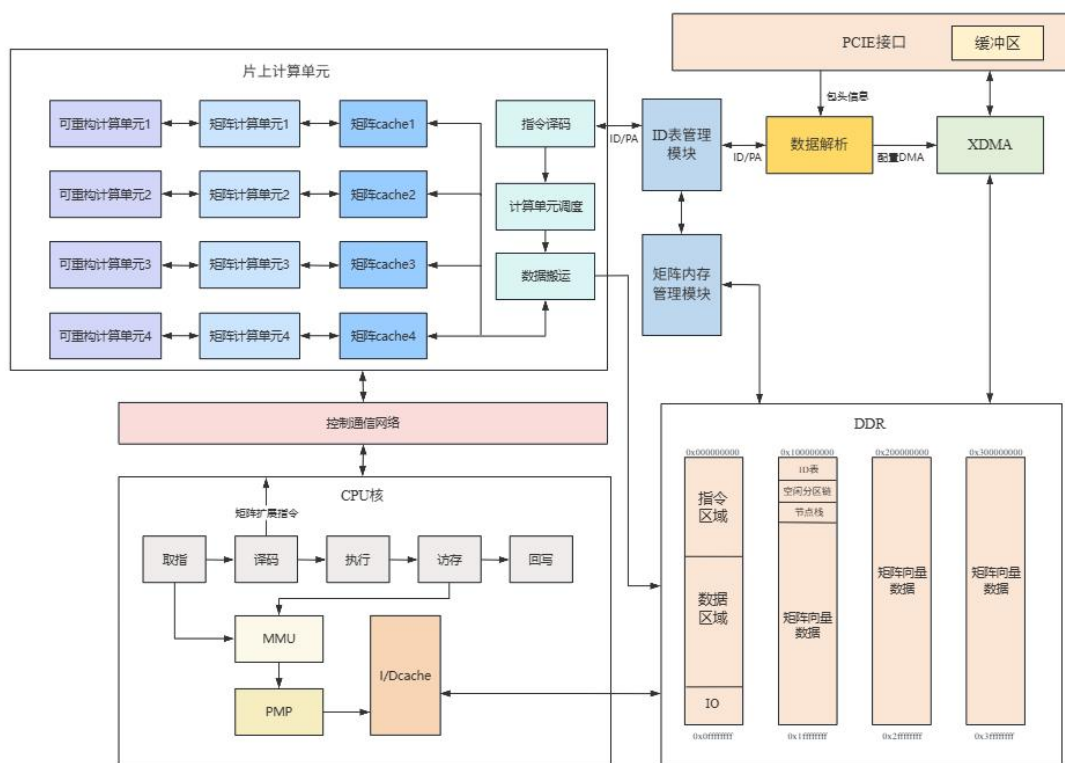


图 15 子板架构图

① 输入输出:

子板作为系统的核心执行载体, 其接口设计围绕七类控制指令与三类状态反馈的双向交互展开,

通过 PCIe 高速接口实现与调度板的高效数据流转。

在输入端，调度板的 PCIe 接口承载四种主要数据类型：首先是资源管理类指令，包括 ID 分配和 ID 释放指令，这些指令采用不定长格式，主要用于矩阵存储空间的开辟与回收；其次是数据传输控制指令，包括矩阵数据传输指令和计算指令传输指令，这些指令采用定长格式，作为数据传输的前置控制信号；第三类是实际载荷数据，包含矩阵数据包和计算指令包，用于进行数据的实际传输；最后是结果交互指令，采用不定长格式，用于触发指定数据的回传。

子板的输出接口遵循状态反馈的设计逻辑。通过 PCIe 向调度板发送的信息包含三类：一是周期性（1s/次）的资源剩余情况，包含已完成计算 ID 列表、内存剩余容量及计算任务剩余数，为调度板提供实时资源调度依据；二是事件触发式的异常情况报告，分为传输异常（如校验失败、超时等）与计算异常（如维度不匹配、运算溢出等）；三是响应结果返回指令的结果数据包，以 4KB 为传输单元，包含对应 ID 的计算结果数据及必要的包头信息。

整个交互过程以 4KB 为基础传输单元，指令与数据均采用统一封装协议，通过 PCIe 接口的低延迟特性实现指令流与数据流的高效协同，保障系统的精准执行与调度。

② 模块设计：

i. 数据接收模块

该模块通过 PCIe 接口接收调度板数据，解析包头识别指令类型后执行相应处理：针对 ID 分配指令和 ID 释放指令，触发中断并由软件完成内存空间分配和释放；处理矩阵数据传输指令时，提取包数后按 4KB 格式解析后续数据并配置 DMA；对于数据传输数据包，解析包头信息，通过 TAT（张量分配表，Tensor Allocation Table）获取物理地址并通过 DMA 实现 PCI 与 DDR 的大量数据交换；指令传输数据包通过 DMA 完成数据交互后触发中断驱动子板矩阵计算；结果返回指令经解析后，获取目标地址并通过 DMA 回传数据至调度板；

ii. CPU 模块

本设计采用五级流水线架构 CPU，完整涵盖取指、译码、执行、访存、写回五个经典阶段，并集成 MMU（内存管理单元）与 PMP（物理内存保护）模块以保障内存访问的安全性及高效性。指令执行逻辑采用差异化处理机制：对于非矩阵计算类指令，由 CPU 按常规流水线流程完成全周期处理；针对矩阵计算指令，其采用 4B 固定长度设计，在取指阶段即启动预解析，若判定为矩阵计算扩展指令，则打包为微码，通过片上控制通信网络传输至专用指令译码单元进行后续处理，实现异构计算任务的精准分流。

iii. 片上控制通信网络

片上网络的拓扑结构决定了核心间的连接方式和数据流的路径。本设计的片上网络采用二维网格拓扑与环形拓扑相结合的方式，确保在大规模核集群中提供灵活的通信路径和高效的数据传输。二维网格拓扑所有核心均通过二维网格连接，每个核心与其上下左右的邻接核心直接相连。这种结构适合大规模并行计算任务，并且具有较好的扩展性和均衡性。环形拓扑则适用于数据传输路径较短、通信延迟要求较低的场景。核心通过环形连接进行通信，能够确保数据的快速传递，尤其适合高频次的小数据包传输。

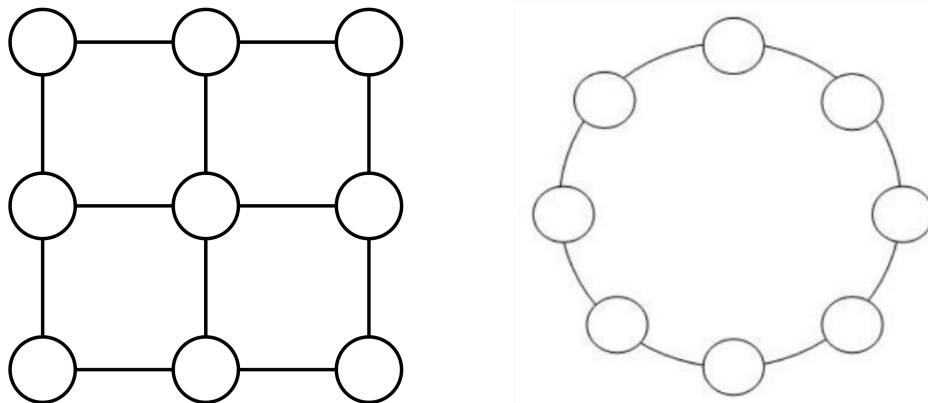


图 16 二维拓扑与环形拓扑示意图

为了保证通信效率的同时降低延迟，本设计结合了二维网格和环形拓扑的优点，设计了一种混合拓扑结构。每个网格节点（核心）与相邻的上下左右核心直接连接，可以通过 X-Y 路由算法来传输数据。环形拓扑通常用于全局同步任务。在实际设计中，每个核心通过环形链路与其他核心相连接，数据可以通过顺时针或逆时针的方式进行传输。

iv. 矩阵计算模块

本设计聚焦 256×256 维度矩阵的高效计算，采用 COO（坐标列表）格式存储矩阵数据，矩阵计算模块支持 16 类核心运算指令，覆盖向量操作、矩阵操作及异构计算场景，具体功能如下：

表 4 计算指令表

指令名称	含义
向量乘法指令	完成 2 个向量的乘法运算
向量加法指令	完成 2 个向量的加法运算
向量求均值	计算向量中所有元素的平均值
向量除法	对向量的每个元素执行除法操作
矩阵转置	将矩阵的行和列交换
矩阵对角化	将矩阵转化为对角矩阵

向量合并	将多个向量合并为一个向量
矩阵乘法指令	完成 2 个矩阵的乘法运算
矩阵加法指令	完成 2 个矩阵的加法运算
矩阵向量乘法指令	完成矩阵和向量的加法运算
数乘向量指令	完成数据和向量的乘法运算
数乘矩阵指令	完成数据和矩阵的乘法运算
向量按位乘法	对两个向量的相应元素执行按位乘法操作
激活函数	对向量或矩阵中的每个元素应用激活函数
矩阵合并	将多个矩阵按行或按列拼接成一个更大的矩阵

v.可重构计算模块

可重构接口模块作为计算任务的调度入口，负责接收来自矩阵模块的 4 路计算请求，通过内置仲裁机制对并发请求进行优先级排序后，将任务存入缓冲池，同时向请求源返回应答信号以启动数据接收流程。待数据完整接收并确认就绪后，接口模块将封装好的微码指令提交至可重构配置模块，由配置模块通过解析预配置字生成适配任务需求的计算模式，随后通过片上搜索网络扫描可用算子子阵列并完成区域锁定。若当前算子资源不足，则进入循环等待状态直至获取足量资源，确保计算资源的精准匹配。

可重构计算阵列作为核心执行单元，包含 4 个 PE 子阵列（3 个浮点运算阵列适配高精度计算、1 个整型阵列支持低延迟操作），其硬件结构采用分层设计：每 4 个 PE 组成独立子网，子网顶层的超级 PE 集成乘法器等复杂运算单元，子网内部通过专用数据通路实现逐层传递，且每个 PE 均配备内部缓存以减少数据搬运开销，提升局部数据复用效率。在阵列控制模块的统一调度下，PE 子阵列按预设计算模式完成并行运算后，最终结果通过专用数据通路回传至矩阵计算模块，形成“请求-调度-计算-反馈”的完整闭环。

vi. TMU 矩阵管理单元

TMU（Tensor Management Unit）是整个矩阵数据管理系统的核心控制枢纽和硬件执行单元。它的核心使命是为上层计算单元（如微码控制单元）提供高效、安全的矩阵数据访问服务。具体而言，TMU 承担着矩阵物理内存管理的核心职责，其核心功能是接收来自微码控制单元或其他请求源的特定矩阵 ID，并基于此 ID 精准地定位到该矩阵在物理内存中的实际位置。实现这一点的关键在于 TMU 维护并管理着 MAT（矩阵分配表）这一核心数据结构。TMU 通过查询 MAT 中与该矩阵 ID 对应的条目（MAT Entry），提取出该矩阵在内存中的物理基地址及其尺寸信息，并将这些关键信息返回给

请求者，为后续的数据加载或存储操作奠定基础。TMU 不仅仅提供地址转换功能，也管理着矩阵的访问控制权限以及安全性。在查询 MAT Entry 的过程中，TMU 会同步检查该条目中预设的访问权限控制位。这些权限位定义了哪些操作（如读、写、特定执行上下文访问等）是被允许的。TMU 会严格验证当前访问请求的权限是否符合 MAT Entry 的规定，进行实时的访问权限检测。如果请求违反了权限规则，TMU 将阻止访问并可能触发异常或错误信号，从而实现对敏感矩阵数据的访问控制和安全性保护，防止未授权或越权操作破坏数据完整性或机密性。因此，TMU 集矩阵物理地址映射、访问权限强制实施、数据安全保护以及 MAT 表维护管理（如响应分配/释放请求更新表项）等关键功能于一身。

	矩阵基地址(24bit)	矩阵大小(9bit)	进程标识符(ASID)	权限位(R W)	Global(1bit)	结果位(1bit)	Valid(1bit)
ID 0							
ID 1							
⋮							
ID 2 ²⁴⁻¹							

图 17 MAT 表项图

MAT (Matrix Allocation Table) 是存储在系统内存中特定区域的一张全局性元数据表，它是 TMU 进行矩阵管理和访问控制的唯一权威信息来源。这张表的结构化设计使其能够系统化地记录系统中所有已知矩阵的关键属性和管理信息。每个矩阵在 MAT 中都拥有一个唯一的条目 (MAT Entry)，该条目是矩阵管理的最小单位。一个 MAT Entry 包含了多个核心字段：矩阵 ID 作为全局唯一的标识符，是 TMU 查找条目的关键索引；物理基地址精确指明了该矩阵数据块在物理内存中的起始位置；矩阵大小定义了矩阵的空间范围，使 TMU 和请求者能够理解数据的边界；权限控制位是 MAT 的核心安全属性，它们编码了该矩阵允许的访问模式及权限。MAT 的本质是一个静态或动态更新的配置数据库，它为每个矩阵建立了从逻辑 ID 到物理内存布局以及安全策略的映射关系，是系统对矩阵资源进行统一管理、隔离和保护的基础设施。TMU 的几乎所有核心功能都依赖于准确和最新的 MAT 信息，MAT 具体结构如图 17 所示。

vii DDR 内存模块

板载 64GB DDR4 内存 (4bank×16GB) 被精心划分为多个功能分区，旨在优化系统的性能与资源利用，确保各类操作的高效进行。具体分区如下：

指令区域：该区域用于存储系统运行所必需的各类指令，包括控制指令、数据传输指令、计算指令等。这些指令决定了整个计算流程的调度、执行和资源分配，保障了系统的高效运行。通过将

指令区域与其他数据区域分开存储，可以有效避免指令与数据的相互干扰，提高指令获取速度，减少访问延迟。

数据区域：此区域专门用于存储 CPU 访存所需的数据。包括各种缓存数据、计算所需的中间结果以及用于计算任务的输入数据。

IO 区域：此区域专门用于存储外设及其他设备的数据，如传感器数据、输入输出缓存以及其他外部通信的数据。

TAT 表：该表存储矩阵 ID 与物理地址之间的映射关系。它是系统资源管理的关键部分，通过将矩阵数据与其物理位置进行映射，确保系统能够快速定位所需的数据。

空闲分区链/节点栈：此区域记录物理内存地址的空闲节点和分区状态，以便于内存的动态分配和回收。空闲分区链的管理能够保证内存资源的高效利用，避免内存碎片化问题，提升内存的整体使用率。此外，节点栈提供了对内存分配的快速响应机制，支持系统的实时调度需求。

矩阵数据区：此区域专门用于存储矩阵计算所需的矩阵和向量数据。这些矩阵和向量通常是计算过程中必不可少的数据元素，存储在这一区域内可以确保快速访问，并支持复杂的数学计算操作。矩阵数据区的设计强调对大规模数据的处理能力，能够高效地支持并行计算和数据传输需求。

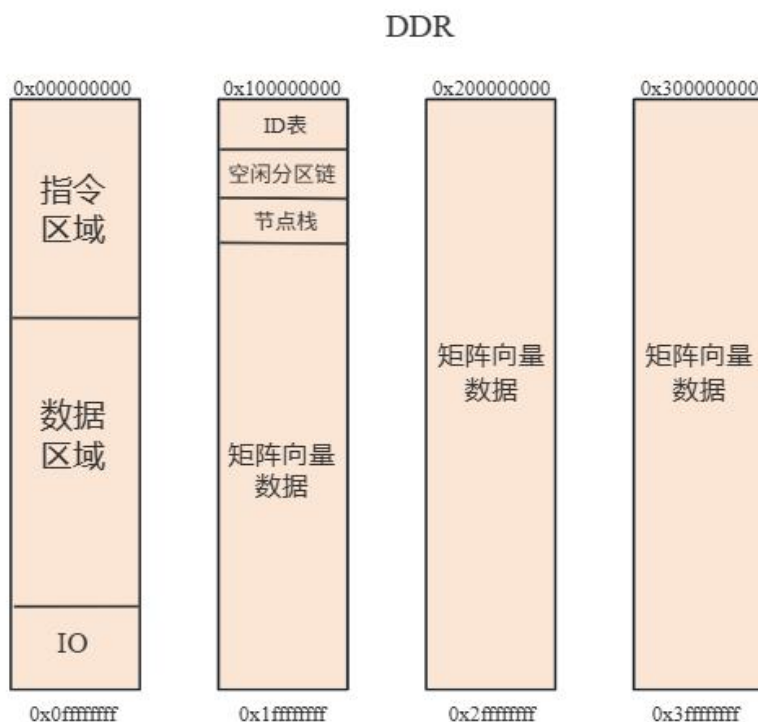


图 18 DDR 功能分区图

③ 工作流程：

接收数据：通过 PCIE 接口接收到数据后，分析包头得到指令类型。

初始时，接收 ID 分配指令，解析出 ID 数量和 ID 号，产生异常或者中断，异常中断处理程序为矩阵开辟物理存储空间并管理表项，开辟完成后返回给调度板完成信号。

ID 开辟完成后，接收到数据传输指令，得到传输包数，接下来的数据以 4KB 包的数据包格式解析。

接收到数据传输包数据，解析包头数据，得到矩阵 ID、包数量、包索引、数据大小。当包索引为 0 时，将 ID 发送给 ID 管理表，得到对应的物理地址，结合包头信息配置 DMA，进行数据传输，当所有数据包传输完成后返回给调度板完成信号。

数据接收完成后，接收到指令传输指令，得知指令包数，配置 DMA，接下来的包以 4KB 的指令包进行解析。

接收到指令包后，通过 DMA 进行数据传输，指令包全部传输完成后返回给调度板完成信号并产生中断，中断返回地址设置为指令区域的基址。这时，矩阵数据和指令接收完毕，可以开始进行计算。

CPU 启动后，译码模块进行指令解析，在解析出矩阵扩展指令时，访问寄存器获取矩阵 ID，然后将该 ID 通过控制网络传送至微码处理单元。如果指令不是矩阵扩展指令，则按照常规的指令执行流程进行，即“取指-译码-执行-访存-写回”。

微码处理单元解析指令并且进行计算资源调度，得到计算单元 ID、矩阵 ID、计算类型，之后将矩阵 ID 发送给 ID 表模块，得到对应的物理地址，然后将数据块从 DDR 搬运到 SRAM，驱动矩阵计算模块进行运算。

四个矩阵计算模块并行计算，可能存在相关性，如果该条指令有相关性，那么该条计算指令被阻塞，微码处理单元可以缓存多条指令，如果该条指令被阻塞，那么下一条没有相关性的指令进行执行。

矩阵计算完成后，矩阵计算模块会将完成信号返回给微码控制单元，微码控制单元会产生中断或者挑选下一条计算指令继续进行运算。

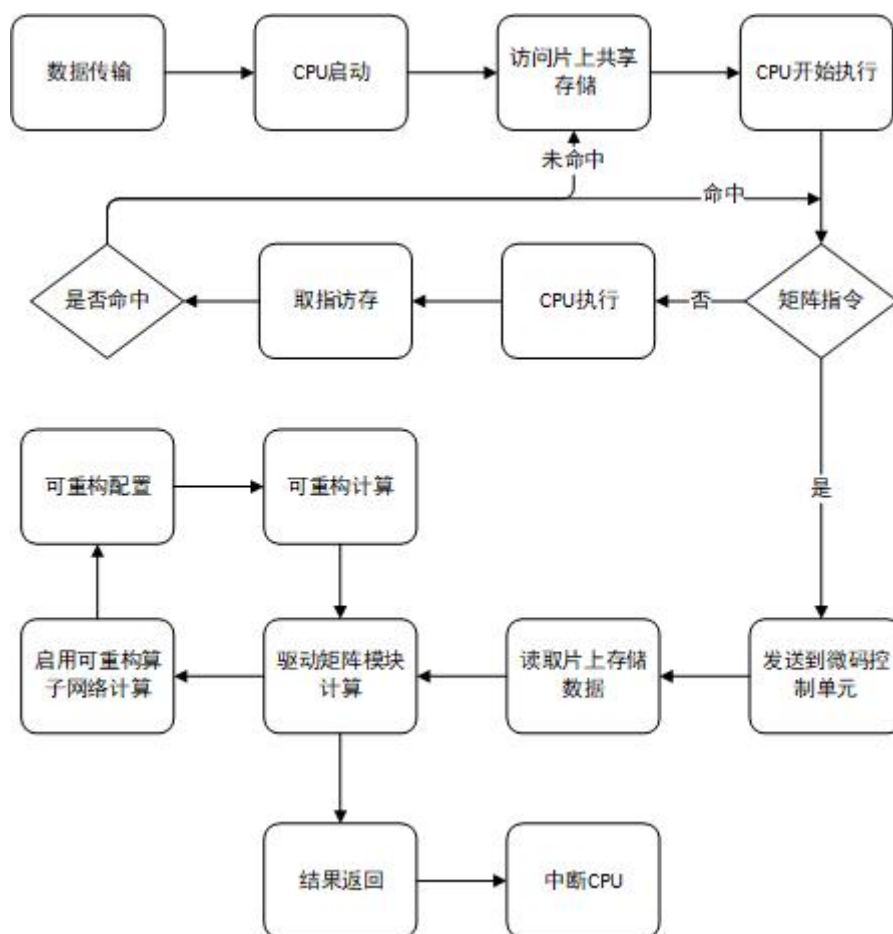


图 19 子板工作流程图

4. 本发明创造的优点（请填写）：

针对上述“现有技术的缺陷和不足”并结合技术方案中的各个发明改进点作出具体说明，即以推理方式具体分析各个改进点如何带来这些优点，使得对优点的说明做到有理有据；它可以是成本低、结构简单、性能好、使用方便及生产质量高等。

本方案通过三级架构创新实现系统性突破：在传输范式重构层面，采用分离式传输协议打破“指令-数据”绑定困局，通过数据预压缩（COO 格式）与 ID 寻址机制从源头消除冗余传输；在调度机制革新层面，依托硬件级元数据表动态追踪系统状态，结合数据局部性优化原则实现纳秒级任务分配；在可靠性架构升级层面，通过分层容错机制（指令重试→板级隔离→系统迁移）构建完整弹性体系，实现故障自愈无需人工干预；在计算连续性保障层面，采用双通道 DMA 流水线配合环形缓冲区，确保数据传输与计算操作完全解耦，最终形成从通信优化、资源调度、容错恢复到计算并行的全栈式技术闭环。

5. 附图：

附图中的各零部件名称应该用标号或名称标注，尽可能的提供可编辑格式的文本，如 CAD 格式。
可通过多个附图来展示（整体图、局部图、外部图、内部图、侧视图等）

例如：

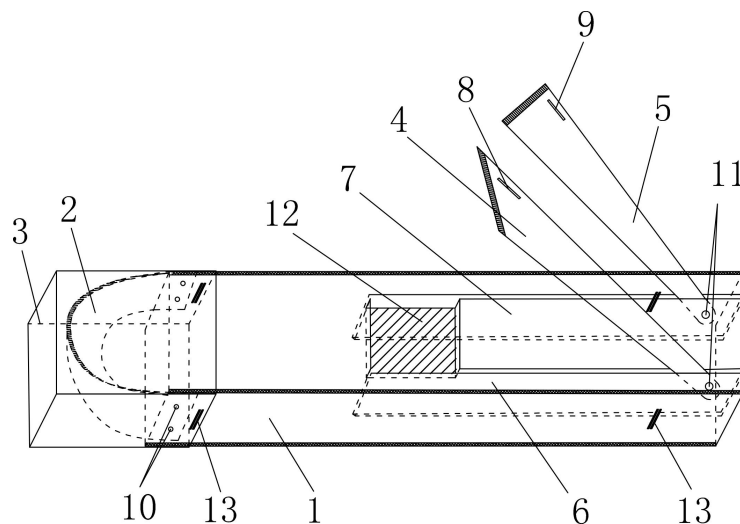


图 1：果树腐烂病刮治器的结构示意图

1、手柄； 2、弧形刀体； 3、刀体盖； 4、斜口刀体； 5、平口刀体； 6、斜口刀鞘； 7、平口刀鞘；
8、斜口刀开启槽； 9、平口刀开启槽； 10、固定钉； 11、转动轴； 12、拉刀槽； 13、扣钮。